

2.1 Write a program to implement a circular queue and show the following: insertion and deletion.

```
class CircularQueue {  
    int[] queue;  
    int front = -1, rear = -1, size;  
  
    CircularQueue(int size) {  
        this.size = size;  
        queue = new int[size];  
    }  
  
    void insert(int value) {  
        if ((rear + 1) % size == front) {  
            System.out.println("Queue is Full");  
            return;  
        }  
        if (front == -1)  
            front = 0;  
        rear = (rear + 1) % size;  
        queue[rear] = value;  
        System.out.println(value + " inserted");  
    }  
  
    void delete() {  
        if (front == -1) {  
            System.out.println("Queue is Empty");  
            return;  
        }  
    }  
}
```

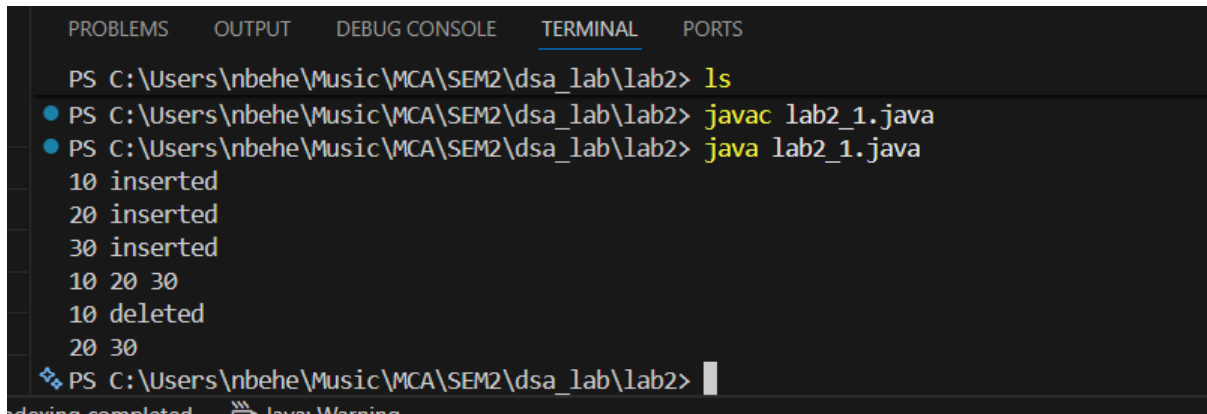
```
System.out.println(queue[front] + " deleted");  
if (front == rear) {  
    front = rear = -1;  
} else {  
    front = (front + 1) % size;  
}  
}
```

```
void display() {  
    if (front == -1) {  
        System.out.println("Queue is Empty");  
        return;  
    }  
    int i = front;  
    while (true) {  
        System.out.print(queue[i] + " ");  
        if (i == rear)  
            break;  
        i = (i + 1) % size;  
    }  
    System.out.println();  
}
```

```
public static void main(String[] args) {  
    CircularQueue q = new CircularQueue(5);  
    q.insert(10);  
    q.insert(20);  
    q.insert(30);
```

```
    q.display();  
    q.delete();  
    q.display();  
}  
}
```

Output:



```
PS C:\Users\nbehe\Music\MCA\SEM2\dsa_lab\lab2> ls  
PS C:\Users\nbehe\Music\MCA\SEM2\dsa_lab\lab2> javac lab2_1.java  
PS C:\Users\nbehe\Music\MCA\SEM2\dsa_lab\lab2> java lab2_1.java  
10 inserted  
20 inserted  
30 inserted  
10 20 30  
10 deleted  
20 30  
PS C:\Users\nbehe\Music\MCA\SEM2\dsa_lab\lab2>
```

2.2 Write a program to implement Linear Linked List and show the following operations: creation, display, insertion, deletion and searching. class LinearLinkedList {

```
class Node {  
    int data;  
    Node next;  
    Node(int data) {  
        this.data = data;  
    }  
}  
  
Node head;  
  
void insert(int data) {  
    Node newNode = new Node(data);  
    if (head == null) {  
        head = newNode;  
    }  
}
```

```
        return;
    }
    Node temp = head;
    while (temp.next != null)
        temp = temp.next;
    temp.next = newNode;
}

void display() {
    Node temp = head;
    while (temp != null) {
        System.out.print(temp.data + " ");
        temp = temp.next;
    }
    System.out.println();
}

void delete(int key) {
    Node temp = head, prev = null;
    if (temp != null && temp.data == key) {
        head = temp.next;
        return;
    }
    while (temp != null && temp.data != key) {
        prev = temp;
        temp = temp.next;
    }
    if (temp == null) return;
```

```
        prev.next = temp.next;
    }

    void search(int key) {
        Node temp = head;
        while (temp != null) {
            if (temp.data == key) {
                System.out.println("Found");
                return;
            }
            temp = temp.next;
        }
        System.out.println("Not Found");
    }

    public static void main(String[] args) {
        LinearLinkedList list = new LinearLinkedList();
        list.insert(10);
        list.insert(20);
        list.insert(30);
        list.display();
        list.search(20);
        list.delete(20);
        list.display();
    }
```

```
}
```

```
PS C:\Users\nbehe\Music\MCA\SEM2\dsa_lab\lab2> javac lab2_2.java
PS C:\Users\nbehe\Music\MCA\SEM2\dsa_lab\lab2> java lab2_2.java
10 20 30
Found
10 30
PS C:\Users\nbehe\Music\MCA\SEM2\dsa_lab\lab2> 
Indexing completed. Java: Warning
```

2.3 Write a program to implement a Doubly Linked List and show the following operations: creation, display, insertion, deletion, and searching. DoublyLinkedList

```
class DoublyLinkedList {
```

```
    class Node {
```

```
        int data;
```

```
        Node prev, next;
```

```
        Node(int data) {
```

```
            this.data = data;
```

```
        }
```

```
    }
```

```
    Node head;
```

```
    void insert(int data) {
```

```
        Node newNode = new Node(data);
```

```
        if (head == null) {
```

```
            head = newNode;
```

```
            return;
```

```
        }
```

```
        Node temp = head;
```

```
        while (temp.next != null)
```

```
            temp = temp.next;
```

```
        temp.next = newNode;
```

```
    newNode.prev = temp;
}
```

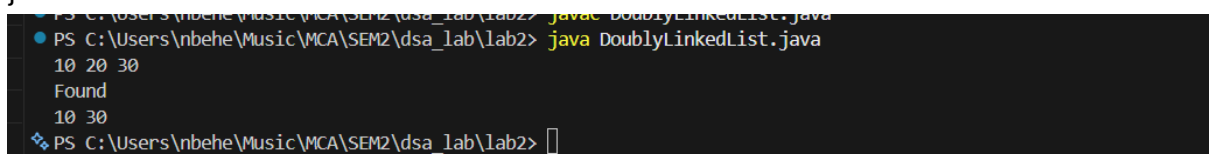
```
void display() {
    Node temp = head;
    while (temp != null) {
        System.out.print(temp.data + " ");
        temp = temp.next;
    }
    System.out.println();
}
```

```
void delete(int key) {
    Node temp = head;
    while (temp != null && temp.data != key)
        temp = temp.next;
    if (temp == null) return;
    if (temp.prev != null)
        temp.prev.next = temp.next;
    else
        head = temp.next;
    if (temp.next != null)
        temp.next.prev = temp.prev;
}
```

```
void search(int key) {
    Node temp = head;
    while (temp != null) {
```

```
        if (temp.data == key) {  
            System.out.println("Found");  
            return;  
        }  
        temp = temp.next;  
    }  
    System.out.println("Not Found");  
}
```

```
public static void main(String[] args) {  
    DoublyLinkedList list = new DoublyLinkedList();  
    list.insert(10);  
    list.insert(20);  
    list.insert(30);  
    list.display();  
    list.search(30);  
    list.delete(20);  
    list.display();  
}
```



```
PS C:\Users\nbehe\Music\MCA\SEM2\dsa_lab\lab2> java DoublyLinkedList.java  
10 20 30  
Found  
10 30  
PS C:\Users\nbehe\Music\MCA\SEM2\dsa_lab\lab2>
```

2.4 Write a program to implement a binary tree and display the contents using preorder, postorder, and in-order traversal techniques. class BinaryTree {

```
class Node {  
    int data;  
    Node left, right;  
    Node(int data) {
```



```
        this.data = data;
    }
}
```

```
Node root;
```

```
void insert(int data) {
    root = insertRec(root, data);
}
```

```
Node insertRec(Node root, int data) {
    if (root == null) {
        root = new Node(data);
        return root;
    }
    if (data < root.data)
        root.left = insertRec(root.left, data);
    else
        root.right = insertRec(root.right, data);
    return root;
}
```

```
void inorder(Node root) {
    if (root != null) {
        inorder(root.left);
        System.out.print(root.data + " ");
        inorder(root.right);
    }
}
```

```
}
```

```
void preorder(Node root) {  
    if (root != null) {  
        System.out.print(root.data + " ");  
        preorder(root.left);  
        preorder(root.right);  
    }  
}
```

```
void postorder(Node root) {  
    if (root != null) {  
        postorder(root.left);  
        postorder(root.right);  
        System.out.print(root.data + " ");  
    }  
}
```

```
public static void main(String[] args) {  
    BinaryTree tree = new BinaryTree();  
    tree.insert(50);  
    tree.insert(30);  
    tree.insert(70);  
    tree.insert(20);  
    tree.insert(40);  
  
    System.out.print("Inorder: ");  
    tree.inorder(tree.root);
```

```
System.out.println();
```

```
System.out.print("Preorder: ");
```

```
tree.preorder(tree.root);
```

```
System.out.println();
```

```
System.out.print("Postorder: ");
```

```
tree.postorder(tree.root);
```

```
}
```

```
}
```

```
● PS C:\Users\nbehe\Music\MCA\SEM2\dsa_lab\lab2> java lab2_4.java
Inorder: 20 30 40 50 70
Preorder: 50 30 20 40 70
Postorder: 20 40 30 70 50
❖ PS C:\Users\nbehe\Music\MCA\SEM2\dsa_lab\lab2>
dexing completed. Java: Warning
```